RESEARCH PAPER IMPLEMENTATION REPORT

Implementation of Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity

Based on the research paper published in NAACL 2024

Submitted by

Yadnesh Tayade
PRN: 202401110057
Batch: A3

Department of Computer Science and Engineering
(Artificial Intelligence and Machine Learning)

MIT Academy of Engineering, Alandi

Academic Year: 2026–2027

Abstract

Retrieval-Augmented Generation (RAG) combines a language model with information retrieved from an external knowledge source. A fixed retrieval strategy may perform unnecessary retrieval for simple questions or provide insufficient evidence for complex questions. Adaptive-RAG addresses this issue by selecting a question-answering strategy according to the complexity of the input query. The original framework considers three strategies: answering without retrieval, single-step retrieval, and iterative multi-step retrieval.

This report studies the proposed methodology and presents a compact Python implementation in Google Colab to demonstrate its central routing concept. The Colab output reports routing accuracy of 0.40 on five test examples, an average of 1.6 retrieval steps, and approximately 0.0013 seconds average execution time for that routing test. A separate three-question visualization demonstrates the distribution of the three strategies. Its execution-time values are illustrative and are not measured model runtimes. Therefore, this implementation demonstrates the routing workflow rather than reproducing the full research system.

Keywords: Retrieval-Augmented Generation, Adaptive-RAG, question complexity, retrieval, language models, Python, Google Colab.

1. Introduction

Large Language Models (LLMs) are used for question answering, summarization, and text generation. However, they may generate incorrect or outdated information when relying only on knowledge acquired during training. RAG reduces this limitation by retrieving relevant information from an external collection and supplying it as context to the language model.

A conventional RAG pipeline often applies the same retrieval procedure to every question. This is not always efficient: a straightforward question may be answered without retrieval, a question requiring one supporting fact may need one retrieval operation, and a complex multi-hop question may require several retrieval and reasoning steps.

1.1 Problem Statement

Using multi-step retrieval for every query can increase processing cost and response time. Conversely, using only single-step retrieval may not provide enough evidence for questions that require information from multiple documents. Adaptive-RAG addresses the problem of selecting an appropriate question-answering strategy based on the complexity of each input question.

1.2 Objectives

The objectives are to study the selected paper; understand its methodology and architecture; implement the core adaptive-routing concept in Python and Google Colab; generate and inspect experimental outputs; and identify limitations and possible improvements.

2. Research Paper and Proposed Methodology

The selected paper is "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity" by Soyeong Jeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang, and Jong C. Park. It was published in the Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL), pages 7036–7050.

2.1 Three Question-Answering Strategies

A. No Retrieval: For a simple question, the language model generates an answer directly, without retrieving external documents. The paper expresses this as a = LLM(q), where q is the input question.

B. Single-Step Retrieval: For a moderate question, the system retrieves relevant document(s) once and supplies them to the language model. This can be represented as d = Retriever(q; D), followed by a = LLM(q, d), where D is the document collection and d is the retrieved context.

C. Multi-Step Retrieval: For a complex question, retrieval and generation can be repeated. Previously retrieved information can be included in the context for subsequent steps. This supports questions that require combining evidence from multiple documents.

2.2 Complexity Classification

The original framework uses a smaller language-model-based classifier to assign one of three categories: A (simple), B (moderate), or C (complex). The predicted category determines the retrieval strategy. The paper describes automatically collecting training labels by running different answering strategies and selecting the simplest strategy that answers correctly. If the strategies fail, characteristics of the source dataset are used as a fallback for assigning a label. A classifier is then trained on question–complexity pairs.

2.3 Architecture

The high-level architecture consists of an input question, a complexity classifier, a routing decision, and the selected answering strategy. The output of the selected strategy is passed to the language model to generate the final answer.

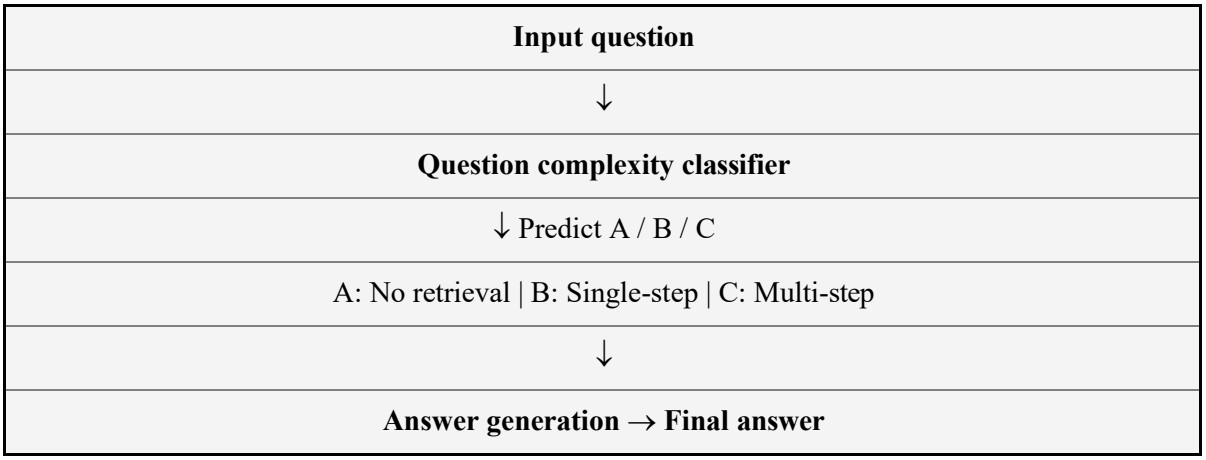


Figure 1. High-level Adaptive-RAG routing architecture.

3. Implementation

3.1 Tools and Environment

The prototype was implemented in Python using Google Colab. Pandas was used to organize tabular results, Matplotlib to create plots, and a classification report was used to inspect the routing predictions.

Component	Technology
Programming language	Python
Development platform	Google Colab
Tabular data handling	Pandas
Visualization	Matplotlib
Classification evaluation	scikit-learn

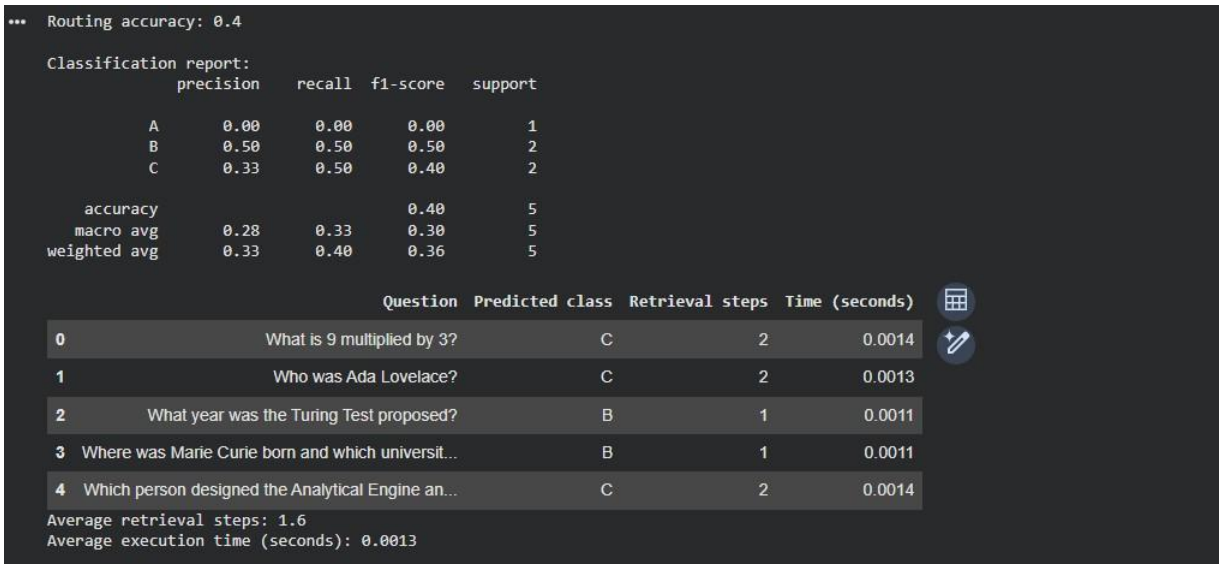
3.2 Implementation Workflow

The notebook prepares a small set of questions and associates each with a complexity class. Class A maps to no retrieval, class B maps to single-step retrieval, and class C maps to multi-step retrieval. The implementation records predicted classes, retrieval-step counts, and execution times. It then compares predicted classes with available test labels and displays the results in a table and graphs.

3.3 Complexity-to-Strategy Mapping

Class	Complexity	Selected strategy
A	Simple	No Retrieval
B	Moderate	Single-Step Retrieval
C	Complex	Multi-Step Retrieval

3.4 Colab Implementation Screenshot



```
Routing accuracy: 0.4

Classification report:
              precision    recall  f1-score   support

           A       0.00      0.00      0.00         1
           B       0.50      0.50      0.50         2
           C       0.33      0.50      0.40         2

    accuracy                           0.40         5
   macro avg       0.28      0.33      0.30         5
weighted avg       0.33      0.40      0.36         5
```

	Question	Predicted class	Retrieval steps	Time (seconds)
0	What is 9 multiplied by 3?	C	2	0.0014
1	Who was Ada Lovelace?	C	2	0.0013
2	What year was the Turing Test proposed?	B	1	0.0011
3	Where was Marie Curie born and which universit...	B	1	0.0011
4	Which person designed the Analytical Engine an...	C	2	0.0014

```
Average retrieval steps: 1.6
Average execution time (seconds): 0.0013
```

Figure 2. Colab routing accuracy, classification report, predictions, retrieval steps, and execution time.

4. Results and Analysis

4.1 Routing Classification Results

The uploaded Colab output reports a routing accuracy of 0.40 on a test set of five questions. The class-wise metrics are shown below. The support column indicates the number of test examples for each class.

Class	Precision	Recall	F1-score	Support
A	0.00	0.00	0.00	1
B	0.50	0.50	0.50	2
C	0.33	0.50	0.40	2
Accuracy	—	—	0.40	5
Macro avg	0.28	0.33	0.30	5
Weighted avg	0.33	0.40	0.36	5

The same Colab output reports an average of 1.6 retrieval steps and approximately 0.0013 seconds average execution time for the five-question routing evaluation. This time should be interpreted as the measured routing-code time in that run, not as end-to-end RAG latency unless retrieval and answer generation were included in the timing.

4.2 Strategy Distribution and Timing Demonstration

A separate visualization cell uses three demonstration records, with one question assigned to each strategy. The graph therefore displays one question (33.3%) for each of the three strategies. The plotted values of 0.50, 1.20, and 2.10 seconds are illustrative values from the demonstration data, not measured timings from the classifier evaluation.

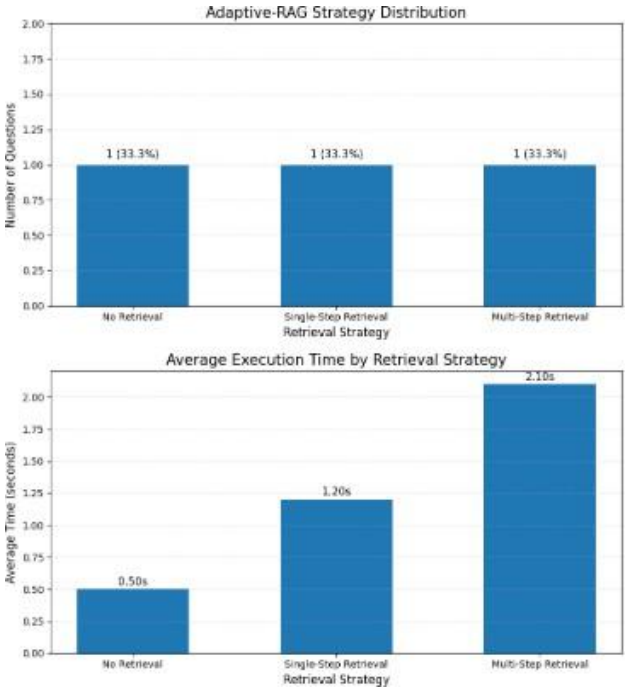


Figure 3. Strategy distribution and illustrative execution-time comparison.

4.3 Colab Result Summary

```
ADAPTIVE-RAG RESULT SUMMARY
--------------------------------

Strategy Distribution:
Selected Strategy
No Retrieval            1
Single-Step Retrieval   1
Multi-Step Retrieval    1
Name: count, dtype: int64

Average Execution Time:
Selected Strategy
No Retrieval            0.5
Single-Step Retrieval   1.2
Multi-Step Retrieval    2.1
Name: Execution Time (sec), dtype: float64

Total Questions: 3
Overall Average Execution Time: 1.267 seconds
```

Figure 4. Summary output for the separate three-question visualization demonstration.

4.4 Interpretation

The routing report indicates that the current classifier demonstration has limited predictive performance: only 40% of the five test labels were predicted correctly. Class A has zero precision and recall in this small test. Since the test set contains only one A example and two examples each for B and C, the reported metrics are highly sensitive to individual predictions. The three-question graph confirms that all three routing paths are represented in the demonstration, but it does not measure classifier accuracy. Similarly, the illustrative timing chart cannot establish a real efficiency advantage.

5. Limitations and Possible Improvements

5.1 Limitations

The evaluation set is very small and has limited class coverage. The compact notebook demonstrates the routing concept but does not reproduce the original paper's complete automatic labelling, classifier training, retrieval, generation, and benchmark evaluation pipeline. The three-question visualization is separate from the five-question classification test. In addition, the displayed strategy timing values are illustrative, and answer quality was not evaluated against reference answers using Exact Match or F1.

5.2 Possible Improvements

The system can be improved by using a larger and balanced dataset with reliable complexity labels; training a dedicated classifier following the paper's labelling approach; integrating a real document collection and retriever; measuring end-to-end retrieval and generation latency over repeated runs; and evaluating answer quality using Exact Match, token-level F1, and human assessment. The adaptive system should also be compared against no-retrieval, single-step, and multi-step baselines.

6. Conclusion

This report studied Adaptive-RAG and implemented a compact demonstration of its central idea: selecting a question-answering strategy according to question complexity. The three strategies considered were no retrieval, single-step retrieval, and iterative multi-step retrieval. The Google Colab output reported 40% routing accuracy on five test examples, an average of 1.6 retrieval steps, and approximately 0.0013 seconds average execution time for the routing evaluation. A separate three-question visualization demonstrated the distribution of the three strategies.

The implementation provides a basic practical understanding of adaptive routing, but it is not a full reproduction of the original research system. The small test set, incomplete retrieval pipeline, and illustrative timing values limit the conclusions that can be drawn. A larger labelled dataset, trained classifier, real retrieval and generation components, and measured end-to-end evaluation are important next steps.

References

[1] S. Jeong, J. Baek, S. Cho, S. J. Hwang, and J. C. Park, "Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity," *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics*, 2024, pp. 7036–7050. Available: https://aclanthology.org/2024.naacl-long.389/

[2] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, 2020.

[3] C. Raffel et al., "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer," *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.